

ENGINEERING HANDOVER PDF EDITION

Production Handover Notes

This chapter defines the honest production boundary of The Ontology Machine.

Production stance: working system, strong architecture, real evidence model. Not yet proven across enough hostile machines, provider failures and long runs.

Contents

Recommended first pass: read the handover position, the production boundary, the fragile paths, and the readiness checklist.

- 12. Production Handover Notes 3**
 - Handover Position 4
 - Already Architected 5
 - Reference Implementation Boundary 6
 - What Is Not Production Hardened 7
 - Security And Auth Boundary 10
 - Cost-Control Boundary 11
 - Provider-Failure Boundary 12
 - Monitoring And Operations Gaps 13
 - Windows Field-Readiness Concerns..... 14
 - Known Fragile Code Paths 15
 - Known Recovery Paths..... 16
 - Recommended Next Hardening Passes 18
 - Production Readiness Checklist..... 22
 - What To Tell The Next Engineer 23
 - Final Boundary..... 24

12. Production Handover Notes

This chapter defines the honest production boundary of The Ontology Machine.

It is not a weakness section. It is the engineering handover line between a working V1 reference implementation and a fully field-hardened product. The Machine can create, ingest, rebuild, merge, query and extend evidence-bound corpus databases. It also has a real modular architecture, a reproducible Artifact Tree, a Corpus DB contract, agent tools, Kernel workflows and a broad test surface.

That does not mean every production failure mode has been exhausted. A system with local runtimes, SQLite databases, file artifacts, provider calls, browser UIs, Windows paths, long-running workflows and LLM-assisted extraction will always have unknown unknowns until it has lived on many machines and many corpora.

The purpose of this chapter is therefore practical:

- name what is already architected
- name what is still reference implementation
- name where production hardening must happen next
- name known fragile paths before they surprise the next engineer
- name the recovery routes that already exist

Handover Position

The current V1 should be handed over as a local Windows-first reference implementation with real working product surfaces.

It is suitable for:

- controlled local demonstrations
- corpus creation and inspection by a technical operator
- sample DB exploration
- custom taxonomy/projection experiments
- Base Graph and ontology lens work
- research-oriented analysis workflows
- further engineering hardening

It should not yet be presented as:

- a managed multi-user server
- a cloud service
- a zero-maintenance desktop product
- a guaranteed unattended multi-day production runner
- a security-audited secrets platform
- an enterprise document pipeline with formal scale limits and SLA

The correct production stance is:

Working system, strong architecture, real evidence model.
Not yet proven across enough hostile machines, provider failures and long runs.

Already Architected

The following parts are not vague future ideas. They exist as product architecture and should be treated as the starting foundation for hardening.

Area	What Exists
Module boundaries	Orchestrator, Optimizer, Interpreter, Validator, Normalizer, Corpus Builder, MCP Server, Semantic Control Kernel, Client Frontend and Edit Suite have distinct ownership.
Artifact Tree	The file system carries originals, page images, model requests, structured output, validation, normalized output, logs, Error Cases, Semantic Release files and Corpus DBs.
Corpus DB	SQLite materialization stores documents, pages, extracted fields/rows, promotions, evidence atoms, embeddings, page images, source-document relations, Base Graph structures and ontology lens data.
Semantic Release	Taxonomy/projection state is versioned, fingerprinted and activated before ingestion.
Evidence chain	Claims can be traced from DB rows to evidence atoms, page images and source artifacts.
Error visibility	Hard failures become Error Cases. Weak or uncertain materializations can stay reviewable instead of being hidden.
Base Graph	Source-document grouping, page ordering and base relations can be built deterministically from materialized identity fields.
Ontology lenses	Lenses can add evidence-bound semantic overlays without overwriting materialized base facts.
Query Agent	Read-only DB tools, compact/full document views, source reconciliation, ontology-aware answers and source viewer integration exist.
Ontology Agent	Ontology/relation-layer writes are separated from base fact mutation and guarded by SQL write policy, preflight, validation and repair loops.
Taxonomy Agent	User-facing access to Kernel workflows exists through a constrained tool surface.
Semantic Control Kernel	Workflow state, dialogs, progress, blocking, resume, recovery and receipts exist outside chat context.
MCP bridge	Kernel workflow tools are exposed to the Frontend agent through a local bridge rather than raw direct module calls.
Config surfaces	Client Frontend and Orchestrator credentials/configuration are separated and documented.
Installer/runtime work	Bundled runtime and startup scripts exist, with source and installed launch paths.
Test architecture	Module-owned tests and cross-module verification surfaces exist, even where production proof is still incomplete.

This is the main reason the system can be hardened instead of merely untangled. The next engineer should preserve these boundaries while fixing production gaps.

Reference Implementation Boundary

The V1 implementation is intentionally local and file-backed.

The normal assumption is:

- one operator
- one Windows machine
- local browser
- local loopback servers
- local Artifact Trees
- local SQLite DBs
- local module runtimes
- external LLM/embedding providers only through configured credentials

The system is not currently designed around:

- concurrent human operators
- remote browser access
- multi-tenant DB separation
- RBAC
- encrypted corpora
- centralized telemetry
- distributed worker queues
- network-file-system guarantees
- formal job scheduling
- automatic provider failover across vendors

That boundary is not a flaw. It is the current product shape. The first production hardening pass should stabilize this local shape before turning it into a broader deployment model.

What Is Not Production Hardened

The following areas are the most important handover risks.

Kernel Workflow And Recovery Hardening

The Kernel owns workflow truth, but its real production behavior still needs more hostile testing.

Known risk classes:

- multi-hour and multi-day workflow continuation
- interrupted owner subprocesses
- sleep/reboot during active workflow
- stale lock proof after process death
- blocked workflow recovery after partial owner mutation
- missing or delayed final notice in the Frontend
- progress event corruption or missing event batches
- resume option lifecycle after external file changes
- support-only blockers that need clearer user wording

The design is already right: workflow state must live in the Kernel, not in the agent transcript. The hardening need is to prove that state under real interruption and long runtime conditions.

Agent-Facing Tool Calls

The Taxonomy Agent has a constrained tool surface, and the Ontology Agent is not allowed to rewrite the base extraction path. This is the correct safe default.

The remaining risk is not the concept. It is tool-path completeness:

- all 16 Taxonomy Agent workflows need real owner smoke runs
- every pending dialog path needs acceptance checks
- every blocked state needs a user-facing explanation
- every resume path needs stale option testing
- every cancel path needs owner cleanup verification
- every final notice needs to appear in the Frontend, not only in logs

Agent-facing workflows should be treated as product API calls, not just internal helpers.

Orchestrator And Pipeline Stage Hardening

The Orchestrator can run the pipeline and preserve stage artifacts, but field behavior still depends on external conditions:

- provider credentials
- provider latency
- OCR request size
- document render size

- unsupported source formats
- locked files
- deep paths
- antivirus delays
- interrupted batch runs
- stale or copied manifests

The main production risk is not that the pipeline has no error handling. It does. The risk is that every stage boundary has many environment-dependent failure modes, and not all of them have been tested on enough machines.

DB, Merge And Rebuild Hardening

The Corpus DB and Artifact Tree model is strong, but DB-scale behavior needs more proof.

Known risk classes:

- large additive merges
- ontology lens preservation during merge
- source-document/base graph preservation during merge
- SQLite variable limits and batching
- WAL/SHM sidecars during interrupted writes
- external tools holding DB locks
- page-image dual-surface consistency
- rebuild from large artifact folders
- vector search over large embedding sets
- non-empty target DBs and partially created merge targets

The merge and rebuild paths are especially important because they touch both the DB and the Artifact Tree. They must remain boring even when the input is not.

Frontend And Browser Hardening

The Frontend has the right surfaces, but browser/UI proof is still not the same as Node/JSDOM proof.

Known risk classes:

- small-screen progress panels
- long Kernel progress displays
- pending dialog visibility
- source list and page image viewer layout
- failed browser auto-launch
- config server vs chat server confusion
- light/dark mode persistence
- source-link suspicion highlighting
- MCP event polling silence when bridge calls fail

The Frontend should expose unhealthy bridge/progress states as visible user states. Silent empty event batches are too easy to misread as “nothing is happening.”

Installer And Deployment Hardening

The installer/runtime work is substantial, but a field-ready desktop product needs repeated install tests on clean machines.

Known risk classes:

- clean Windows user profile
- non-admin install
- missing default browser
- existing process on port 3000 or 3001
- stale server-state files
- deep install path
- spaces and Unicode in paths
- external drive DBs
- locked install files
- antivirus scanning bundled runtimes
- uninstall/update behavior
- sample DB payload registration

A successful install on one other system proves the installer can work. It does not yet prove that it is field-ready across normal Windows chaos.

Security And Auth Boundary

The current security posture is local-operator trust.

Already present:

- local loopback server model
- separated Client Frontend and Orchestrator credential surfaces
- protected config responses that do not simply echo secrets back
- OAuth token storage and refresh behavior
- local MCP bridge visibility rules
- constrained Query/Ontology/Taxonomy agent roles

Not production hardened:

- no formal multi-user auth model
- no RBAC
- no encrypted Corpus DB or Artifact Tree storage
- no full secrets threat model
- no remote access hardening
- no formal OAuth callback allowlist model
- no provider key rotation workflow
- no audit-grade security log
- no red-team review of local HTTP routes

The system should therefore be run as a trusted local desktop application. Do not expose the local HTTP servers to a network. Do not put sensitive corpora on shared machines without an external security model.

Cost-Control Boundary

The Machine is intentionally able to generate many provider calls. That is part of its power, but it is also a production risk.

Cost-sensitive paths:

- OCR/vision extraction
- sample analysis for custom releases
- Interpreter calls
- Normalizer calls
- Kernel LLM-assisted authoring
- embedding generation
- ontology embedding refresh
- large rebuilds with embedding enabled

Current cost controls are mostly operational:

- model configuration is explicit
- request artifacts are persisted where available
- embeddings can be absent without corrupting the DB
- some workflows are bounded by design
- Kernel state can show blocked or completed outcomes

Not yet hardened:

- no global run budget enforced across all modules
- no provider quota dashboard
- no automatic cost stop by project
- no per-workflow token ceiling receipt
- no live cost estimate before a large run
- no rate-limit backoff policy proven across providers

The safe production rule is simple: run large ingests and rebuilds only with known model settings, known embedding settings and enough provider budget.

Provider-Failure Boundary

External providers are a normal failure source, not an exceptional one.

Expected failure classes:

- missing API key
- expired OAuth token
- revoked OAuth token
- billing disabled
- quota exhausted
- rate limit
- transient 5xx
- malformed provider response
- JSON-mode mismatch
- request too large
- model unavailable
- embedding dimension mismatch
- slow response or timeout

The system should degrade where safe. For example, a DB can remain valid without embeddings, and SQL/lexical retrieval can still work. But provider-backed authoring, OCR, normalization and embedding generation cannot be treated as always available.

Next hardening should make provider failure messages more uniform across Frontend, Orchestrator, Kernel and Corpus Builder.

Monitoring And Operations Gaps

The Machine already writes many artifacts, logs, manifests, receipts and DB rows. That is not the same as production monitoring.

Already present:

- stage artifacts
- model request captures
- Error Case bundles
- batch manifests
- Kernel workflow state
- Kernel progress/final notices
- startup logs
- DB integrity queries
- source viewer evidence paths

Missing for production operations:

- central health dashboard
- one-click support bundle
- structured incident timeline
- alerting for stuck workflows
- stale lock dashboard
- provider health dashboard
- cost dashboard
- long-run heartbeat view
- disk-space pressure warnings
- corpus-scale warnings

The current operator can debug the system, but a non-developer user should not be expected to know which log or manifest to inspect first.

Windows Field-Readiness Concerns

Windows is the main target environment and also the main source of field chaos.

The following conditions need dedicated tests:

- `LongPathsEnabled` disabled
- very deep Artifact Tree roots
- very deep install paths
- paths with spaces
- paths with non-ASCII characters
- external drives
- OneDrive/Sync folders
- network shares
- locked SQLite files
- locked image/request artifacts
- antivirus-delayed file writes
- Defender scanning bundled runtimes
- non-admin install
- low disk space
- machine sleep during pipeline run
- reboot during Kernel workflow
- killed browser while server continues
- killed server while browser continues

The safest operator guidance is to use short local paths for large production corpora and avoid live syncing folders while a run is active.

Known Fragile Code Paths

This table is not exhaustive. It names the first paths a developer should inspect when production behavior looks strange.

Area	Fragile Boundary	Why It Matters	First Recovery Direction
Kernel workflows	stale locks, resume state, pending dialogs, final notices	Long-running workflows can be alive, blocked, stale or already completed while the UI still looks similar.	Ask for Kernel status, inspect resume state, use offered recovery/cancel only.
Kernel event bridge	event polling and tool-availability updates	If event listing fails silently, the Frontend can look idle while the Kernel changed state.	Inspect Frontend logs and Kernel state; treat empty events plus stale UI as bridge health issue.
Taxonomy Agent tools	agent-facing workflow calls	These are product workflow entry points and need full owner smoke coverage.	Re-run through workflow dropdown or explicit Kernel status/resume path.
Ontology Agent writes	SQL batch preflight, FK/NOT NULL rules, post-write validation	The agent can create useful lenses, but DB rules are strict and write order matters.	Let the repair loop run; if still blocked, inspect the failed SQL batch and validation result.
Ontology/Query document reads	compact <code>get_document_*</code> views and legacy full reads	Large DBs can burn huge context if the agent jumps straight to full document bundles.	Prefer summary/evidence/rows/provenance views first; inspect logs for repeated full reads.
Query Agent sources	source reconciliation between assistant text and DB source list	Source highlighting is intentionally soft and can reveal hallucinated or leaked sources.	Treat red/unresolved source links as suspicious and verify against the source panel.
Manual Orchestrator runs	progress and final notice	Stage progress can lag or miss the final completion message.	Inspect batch manifest, Error Cases and final artifacts before assuming a hang.
Semantic Release activation	missing or incomplete active release	Ingest must stop if no active release is attached.	Activate or create the release through Kernel/Corpus Builder paths, not manual folder edits.
Corpus DB identity	DB outside selected Artifact Tree <code>Corpus</code> folder	Kernel-governed ingest/reset/rebuild/merge needs target identity, not just any readable DB.	Move/select the valid Artifact Tree target or recreate through the Kernel route.
DB merge	batching, collision maps, ontology/base graph preservation	Merge touches DB rows and copied artifacts; partial targets can be confusing.	Use merge manifests, inspect target DB integrity, do not flatten copied artifacts manually.
SQLite sidecars	WAL/SHM files and locked DBs	Interrupted writes or external DB browsers can leave locks/sidecars.	Close all users of the DB, checkpoint/compact through owner tools where available.
Error Case cleanup	locked artifacts or partial cleanup	Cleanup can leave evidence behind or remove DB records before files disappear.	Keep Error Cases until understood; prefer owner cleanup commands over manual delete.
Batch recovery	latest batch selection	mtime-derived "latest" can be wrong after copy/restore/touch.	Prefer explicit batch IDs when possible.
OCR/request persistence	early failure before request capture	Some failures can happen before a request artifact exists.	Use stage logs plus Error Case bundle; absence of request file is not proof no call was attempted.
Embeddings	missing provider, partial generation, dimension mismatch	Missing embeddings degrade retrieval but do not corrupt the DB.	Check embedding status, provider config and DB embedding tables; regenerate if needed.
Browser launch	default browser, port cleanup, stale state file	Server may start while browser does not, or config server may be mistaken for chat server.	Open the URL manually and inspect startup/browser-helper logs.

Known Recovery Paths

The system already has several recovery routes. Use them before manual surgery.

Kernel Workflow Recovery

Use the Taxonomy Agent or workflow UI to inspect:

```
kernel_status
kernel_resume_state
kernel_continue_resumable_workflow
kernel_cancel_active_run
```

Only continue with a currently offered resume option. Do not invent workflow continuations manually.

Error Case Recovery

Error Cases are diagnostic artifacts, not trash.

Use them to inspect:

- original input
- page images
- model requests
- structured output
- validator output
- normalized output
- stage logs
- failure reason

Only delete Error Cases after deciding they are no longer useful.

DB Rebuild Recovery

When DB state is suspect but Artifact Tree evidence is intact, rebuild from the Artifact Tree instead of hand-editing the DB.

Rebuild relies on:

- active Semantic Release
- normalized artifacts
- sidecars
- page images
- Corpus Builder schema ownership

DB Reset Recovery

Use reset when the Corpus DB should be cleared while preserving the surrounding target structure and release state. Do not delete the SQLite file manually as a substitute for reset.

Merge Recovery

For failed additive merges:

- inspect the target Artifact Tree
- inspect merge run manifests
- inspect collision maps
- inspect whether the target DB is empty, partial or ready
- check whether ontology/base graph rows were copied
- do not manually flatten copied artifact folders

If the merge target was partially created, treat it as a failed run artifact until integrity has been proven.

Embedding Recovery

Missing embeddings are not DB corruption.

Recovery options:

- configure embedding provider
- regenerate corpus embeddings
- refresh ontology embeddings after ontology writes
- accept SQL/lexical retrieval until vectors are available

Credential Recovery

Client Frontend chat credentials and Orchestrator/pipeline credentials are not the same boundary.

If an agent is LLM-ready but a pipeline owner fails, inspect the Orchestrator credential surface and provider role used by that owner.

Startup Recovery

If the browser does not open:

- check whether the server actually started
- check whether the config server or chat server is running
- open the URL manually
- inspect startup logs
- inspect browser-helper logs
- close old processes on the same port if needed

Recommended Next Hardening Passes

The next hardening work should be bounded. Do not try to “harden everything” in one pass. Run focused passes with a written result and a clear stop condition.

Pass 1: Kernel Workflow Gauntlet

Goal: prove that the Taxonomy Agent and Kernel can survive the normal product workflow matrix.

Scope:

- all 16 agent-facing workflows
- pending dialogs
- resume options
- cancel operations
- support-only blockers
- final notices
- stale lock recovery
- owner errors
- missing Semantic Release
- invalid target DB
- interrupted workflow state

Acceptance:

- every workflow has a visible final state
- every blocked state explains ownership and recovery
- no workflow remains as silent “background running” without progress or state
- Frontend and Kernel state agree

Pass 2: Long-Run And Interruption Drill

Goal: prove multi-hour and multi-day behavior.

Scope:

- long manual ingest
- long Kernel ingest
- large OCR render
- large embedding generation
- large merge
- sleep mid-run
- killed browser
- killed server
- killed owner subprocess
- reboot between Kernel states

Acceptance:

- active state is recoverable or safely blocked
- no duplicate owner write starts silently
- stale locks are explainable
- operator knows whether to resume, cancel or inspect support evidence

Pass 3: DB Integrity And Merge Scale Pass

Goal: prove that DB and Artifact Tree operations remain consistent at larger scale.

Scope:

- additive merge with multiple source DBs
- ontology lens preservation
- Base Graph preservation
- source-document classification preservation
- page-image DB/table consistency
- large embedding tables
- WAL/SHM sidecars
- locked DB files
- rebuild from artifacts

Acceptance:

- integrity checks pass
- row counts reconcile
- artifacts copied intentionally
- target DB is either ready or clearly failed
- no partial target is mistaken for production-ready

Pass 4: Provider Failure And Cost Pass

Goal: make provider failure boring and cost visible.

Scope:

- missing key
- expired OAuth
- quota exhausted
- 429
- 5xx
- timeout
- malformed JSON
- model unavailable
- embedding dimension mismatch
- large request rejection

- run budget warnings

Acceptance:

- every provider failure has the same basic message shape
- user sees what degraded and what still works
- no DB is marked corrupt because embeddings are absent
- large runs expose cost-relevant settings before execution

Pass 5: Windows Field Matrix

Goal: prove the Machine on normal hostile Windows setups.

Scope:

- clean user profile
- non-admin install
- short path
- deep path
- path with spaces
- path with non-ASCII characters
- OneDrive/Sync path
- external drive
- locked DB
- locked artifacts
- Defender/antivirus delay
- no default browser
- occupied ports
- uninstall/reinstall

Acceptance:

- startup succeeds or fails with actionable diagnostics
- installer paths are correct
- sample DB and root config are correct
- logs name the real blocker
- no path-length failure appears in normal generated artifact names

Pass 6: Frontend Live Browser Pass

Goal: prove the UI as the user actually sees it.

Scope:

- chat server
- config server
- Query Agent sources
- page image viewer

- ontology/base graph badges
- workflow dropdown
- progress panels
- pending dialogs
- small monitor layout
- light/dark mode
- failed MCP bridge
- failed browser auto-open

Acceptance:

- user can always reach input and progress controls
- bridge failure is visible
- unresolved sources are visibly suspicious
- config changes persist
- server identity is not confusing

Pass 7: Security And Local Exposure Pass

Goal: define and harden the local trust boundary.

Scope:

- local HTTP routes
- config route isolation
- secret masking
- OAuth callback host handling
- token deletion
- MCP bridge token validation
- DB/artifact sensitivity warning
- support bundle redaction

Acceptance:

- no route exposes secrets in normal responses
- config server does not accidentally act like the chat server
- local-only assumptions are documented and enforced where practical
- sensitive support artifacts are either redacted or clearly marked

Production Readiness Checklist

Before handing a build to another operator, capture this checklist.

Check	Expected Result
Frontend starts	Chat UI opens on port <code>3000</code> ; config UI opens on port <code>3001</code> .
Credentials visible	LLM readiness matches actual provider configuration.
DB selected	Active DB path points to the intended Corpus DB.
Artifact Tree valid	DB lives under the selected Artifact Tree <code>corpus</code> folder for Kernel workflows.
Semantic Release active	Ingest target has an active release before pipeline start.
Base Graph known	Badge or DB inspection shows whether Base Graph exists.
Ontology lenses known	Lens count is visible and matches DB inspection.
Source viewer works	A query source opens the expected page image.
Error Cases understood	Error Cases are either absent or intentionally preserved.
Embedding status known	Missing embeddings are documented as degraded retrieval, not corruption.
Kernel status clean	No active/stale workflow remains from previous runs.
Startup logs clean	No stale server-state or port conflict remains.
Small workflow smoke	One small create/ingest/query path completes with a final notice.
Handover notes captured	Known local machine quirks, providers and paths are written down.

What To Tell The Next Engineer

The Machine should not be debugged as one big script.

When something fails, first ask:

Which owner owns this state?

Typical ownership answers:

- Orchestrator owns manual pipeline execution and stage orchestration.
- Optimizer owns render/OCR preparation.
- Interpreter owns structured extraction.
- Validator owns evidence validation.
- Normalizer owns canonical release-shaped output.
- Corpus Builder owns DB schema, materialization, reset, rebuild, merge and embeddings.
- Semantic Control Kernel owns workflow state, dialogs, progress, resume and recovery.
- Client Frontend owns chat/config UI, agent sessions, source panels and visible state.
- MCP Server owns the bridge between Frontend agents and Kernel tools.
- Ontology Agent owns ontology/relation-layer writes only.
- Query Agent owns read-only retrieval and source-backed answering.

Fix the owning boundary. Do not patch around it from a neighboring module unless the boundary itself is wrong.

Also tell the next engineer:

- preserve the Artifact Tree
- preserve Error Cases until understood
- do not hand-edit the DB as a first response
- do not flatten merge artifacts manually
- do not bypass Semantic Release activation
- do not confuse missing embeddings with DB corruption
- do not start duplicate Kernel workflows over stale state
- do not turn ontology lenses into base fact rewrites
- do not hide provider failures behind generic “failed” messages

Final Boundary

The Ontology Machine V1 is a working evidence machine, not a fully hardened industrial deployment.

Its strongest production asset is not that every bug is already known. That would be fantasy. Its strongest asset is that most important state has an owner, most evidence has a place, most failures can become artifacts, and most semantic claims can be traced back to source material.

That is the right shape for a system that still needs hardening.

The next step is not a rewrite. The next step is disciplined field proof:

```
run it on real machines,  
break it deliberately,  
write down what broke,  
harden the owning boundary,  
repeat.
```